

Lab02 - Classic Cryptography - Monoalphabetic Substitution and OTP

Intro

This past week in lecture you should have seen several substitution and shift-by-n encryption systems. This lab will focus on a monoalphabetic substitution cipher. While you won't see these EXACT ciphers used in current cryptography, many of the classic cipher techniques are found in modern day symmetric cryptosystems. And, these classic ciphers provide a good foundation for future examination of current systems.

You **MAY NOT** use an online decryption tool for any part of this lab nor can you ask Chatgpt (or any AI) to write your code for you. You will write your own in python that documents each step in the decryption process (that means writing comments in the code about what you are trying to achieve), and print out the results/output for each step. You may not download someone's code from github and call it yours. Think of these as paper and pencil math homework problems where you must show all your work. You also cannot copy from your friend.

Your Canvas submission will be the **completed template** for the lab which is provided below at the end of this document. **Code** should be labeled `<netid>_Lab02_partXX.py` and submitted as an independent file. Please upload all files to Canvas at the same time. Canvas shows us what you upload by submit button push. If you do 2 or 3 submits, we will only see the very last thing you upload. You don't want us to miss any part of your lab submission.

Part 01

Monoalphabetic substitution ciphers are not secure because they can be subjected to frequency analysis. In our case this would be English language characters that would be analyzed. Below is a ciphertext which you need to convert to plaintext. It was encrypted using a monoalphabetic substitution cipher (each letter in the original text is replaced by another letter. The replacement does not vary), but it **IS NOT** a shift-by-n cipher. The single alphabet used is a permutation of the letters A-Z with a one-to-one mapping.

Again, you can't use an online decryption tool. The submission must include the **lab template** (below), as well as your **code in a separate file** properly named as `<netid>_Lab02_part01.py`

1. [Login to iselab](#) and power on your machine "cpre3310_[netid]_Ubuntu_24.04_desktop"

2. Change into your homework directory. Scp the skeleton code. You should get all the files for both part01 and part02 in two directories. It should create a new lab02 directory with the skeleton code inside. The credentials are repo/repo.

```
cpre3310@cpre3310:~/homework$ scp -r repo@repo.homework.3310.com:/home/repo/lab02 .
```

3. You will need to give yourself executable on the directories and write permissions on these files once they are downloaded. For example
 - a. `chmod 750 partXX`
 - b. `cd partXX`
 - c. `chmod 640 *`
4. In the skeleton code `part01_skel.py` you will need to edit the functions `frequency` and `frequencyAnalysis`. There are TODO comments that provide guidance on how to implement them. Here are some resources that might be helpful if you are not as familiar with Python:
 - `{ }` curly brackets indicate dictionaries. More info:
https://www.w3schools.com/python/python_dictionaries.asp
 - The `.get()` method returns the value of the item with the specified key. More info:
https://www.w3schools.com/python/ref_dictionary_get.asp

To run frequency analysis:

```
python3 part01_skel.py ciphertext.txt
```

The program should read in the `ciphertext.txt` file and count the ciphertext characters in it. It then prints out the frequencies in descending order. The output will look similar to this with the actual frequency analysis of your ciphertext file. The output below is not the same as what you should have.

```
cpre3310@cpre3310:~/homework/lab02/part01$ python3 part01_skel.py ciphertext.txt
Frequency Analysis:
Character      Count  Percentage
I              200    13.54%
C              153    10.36%
R              129    8.73%
Q              114    7.72%
J              111    7.52%
K              104    7.04%
X              103    6.97%
S              79     5.35%
W              77     5.21%
U              71     4.81%
G              55     3.72%
N              51     3.45%
V              34     2.30%
Y              28     1.90%
T              27     1.83%
P              26     1.76%
H              25     1.69%
Z              24     1.62%
L              22     1.49%
O              18     1.22%
F              13     0.88%
B              12     0.81%
M              1      0.07%
```

5. Take a screenshot of your ciphertext character frequencies to include in your lab report.
6. Create a file called `mapping.txt` file. The file format should be `cipherChar:plainChar`, with each pair on a separate line. Use the English character frequency to map the ciphertext characters to the plaintext characters. NOTE: The normal distribution of characters in the English language (descending order) is E, T, A, O, I, N, S, H, R, D, L, C, U, M, W, F, G, Y, P, B, V, K, J, X, Q, Z.
7. Run the program again, this time with a third argument of the `mapping.txt` file.

```
python3 part01_skel.py ciphertext.txt mapping.txt
```

8. The output will look similar to this. Below is just one of many iterations. *You must include **ALL** iterations in your lab report!*

```
cpre3310@cpre3310:~/homework/lab02/part01$ python3 part01_skel.py ciphertext.txt mapping.txt
Decrypted Text:
NIMEUPNIATHGETOERECASAPRHIMECONCAITEDTNGARRYAPRHIMESSWUTSOECNULDOAKETNWEAREALPRHIMESSOETRAKELEDALLNKERTOECNR
LDTNBHIDNIEWUTINCOEREMNULDOEFETCOATOECAITEDTOERECERPRHIMESSESEINUFOWUTHTCASDHBBHMULTNBHIDNUTCOETOERTOEYCE
EREALNIESTOERECASALCAYSSNGETOHIFAWNUTTOEGTOATCASINTASHTSONULDWESNOEMAGEONGEAFAHIAIDCASSADBNROECNULDOAKELHVED
KERYGUMOTNOAKEAREALPRHIMESSNIEEKEIHFATERRHWLESTNRGMAGENITORECASTOUIDERAIDLHFOTIHFIFAIDTOERAHIPNUREDDNCIHTN
RREITSSUDEILYAVINMVHIFCASOEARDATTOEMHTYFATEAIDTOENLDVHIFCEITNNPEIHTHTCASAPRHIMESSSTAIDHIFNUTTOEREHIBRNITNB
TOEFATEWUTFNNDFRAMHNUSCOATASHFOTTOERAHIAIDTOECHIDOADGADEOERLNNVTOECATERRAIDNCIBRNGOEROAHRAIDMLNTOESHTRAIDNCI
HITNTOETNESNBORSONESAIDNUTAFAHIAITTOEOEELSAIDYETSOESAHDTATSOECASAREALPRHIMESSCELLCELLSNNIBHIDTOATNUTTONUFOT
TOENLDJUEEIWUTSOESAHINTOHIFCEITHITNTOEWEDRNGTNNVALLTOEWEDDHIENBBTOEWEDSTEADAIDLAHDAPEANITOEWNNTNGTOEISOETN
NVTCEITYGATTRESSESALDLDHDTOEGNITOEPEAIDTOEITCEITYEHDERDNCIWEDSNITNPNBTOEGATTRESSESNIHSTOEPRHIMESSOADTNLHE
ALLIHFOTHITTOEGNRIHIFSOECASASVEDONCSOEADSLPTNOKERYWADLYSAHDSOEHOAKESMARMELYMLNSEDGYEYESALLIHFOTOEAKEINILYVI
NCSOATCASHITOEWDWUTHCASLYHIFNISNGETOHIFORDSNTOATHAGWLAVIDWLUALLNKERGYWNDYHTSONRRHWLEINCTOEYVIECTOATSOE
CASAREALPRHIMESSWEMAUSESOEADBELTTOPEARHFOTTORNUFOTOETCEITYGATTRESSESALDITOEETCEITYEHDERDNCIWEDSNWYDUTAREA
LPRHIMESSMULDWESSEISHTHKEASTOATSNTOEPRHIMETNNVOERBNROHSCHEBENRINCOEVICTOATOEADAREALPRHIMESSAIDTOEPEACASP
UTHITOEUGUSEUGCOEREHTGAYSTHLLWESEIHBINNIEOASSTNLEIHTTOERETOATHSATRUESTNRY
```

9. This will output your first trial to conduct cryptanalysis on the monoalphabetic substitution cipher. You must copy this iteration (either screenshot or copy and paste the output) into your lab document. *You must show all iterations you complete in finding the final solution of the plaintext. Include a sentence between each iteration explaining the changes.*
10. Review your output and decide if there are words or potential words you could identify. Modify the `mapping.txt` file to try your new character substitutions. Run the program and document this iteration. *Continue the process until you find the plaintext solution.* In this example the final solution would look like the following.

```
cpre3310@cpre3310:~/homework/lab02/part01$ python3 part01_skel.py ciphertext.txt mapping_2.txt
Decrypted Text:
ONCEUPONATIMETHEREWASAPRINCEWHOWANTEDTOMARRYAPRINCESSBUTSHEWOULDHAVETOBEAREALPRINCESSHETRAVELEDALLOVERTHEWOR
LDTOFINDONEBUTNOWHERECOULDHEGETWHATHEWANTEDTHEREWEREPRINCESSESENOUGHBUTITWASDIFFICULTTOFINDOUTWHETHERTHEYWER
EREALONESTHEREWASALWAYSSOMETHINGABOUTTHEMTHATWASNOTASITSHOULDBESOHECAMEHOMEAGAINANDWASSADFORHEWOULDHAVELIKED
VERYMUCHTOHAVEAREALPRINCESSONEEVENINGATERRIBLESTORMCAMEONTHEREWASTHUNDERANDLIGHTNINGANDTHERAINPOUREDOWNTINTO
RRENTSSUDDENLAKNOCKINGWASHEARDATTHECITYGATEANDTHEOLDKINGWENTTOOPENITITWASAPRINCESSSTANDINGOUTTHEREINFRONTOF
THEGATEBUTGOODGRACIOUSWHATASIGHTTHERAINANDTHEWINDHADMADEHERLOOKTHEWATERRANDOWNFROMHERHAIRANDCLOTHESITRANDOWN
INTOTHEOESOFHERSHOESANDOUTAGAINATTHEHEELSANDYETSHESAIIDTHATSHEWASAREALPRINCESSWELLWELL SOONFINDTHATOUTTHOUGHT
THEOLDQUEENBUTSHESAIIDNOTHINGWENTINTOTHEBEDROOMTOOKALLTHEBEDDINGOFFTHEBEDSTEADANDLAIDAPEAONTHEBOTTOMTHENSHE
OKTWENTYMATTRESSESANDLAIDTHEMONTHEPEAANDTHENTWENTYEDERDOWNBEDSONTOPOFTHEMATTRESSESONTHISTHEPRINCESSHADTOLIE
ALLNIGHTINTHEMORNINGSHEWASASKEDHOWSHEHADSLPTOVERYBADLYSAIDSHEIHAVESCARCELYCLOSEDMYEYESALLNIGHTTHEAVENONLYKN
OWSWHATWASINTHEBEDBUTIWASLYINGONSOMETHINGHARDSOTHATIAMBBLACKANDBLUEALLOVERMYBODYITSHORRIBLENOWTHEYKNOWTHATSH
WASAREALPRINCESSBECAUSESHEHADFELTTHEPEARIGHTTHROUGHTHETWENTYMATTRESSESANDTHEWENTYEDERDOWNBEDSNBODYBUTAREA
LPRINCESSCOULDBEASSENSITIVEASTHATSTOHEPRINCELOOKHERFORHISWIFEFORNOWHEKNOWTHATHEHADAREALPRINCESSANDTHEPEAWAS
UTINTHEMUSEUMWHEREITMAYSTILLBESEENIFNOONEHASSTOLENITTHERETHATISATRUESTORY
```

11. Record the plaintext message in your lab report.
12. Record the key (the mapping of ciphertext character to plaintext character) in your lab report.
13. Beside the English language frequency of each character what else could you have calculated to help you find the plaintext? We discussed this in class. If you are having a really difficult time finding the plaintext you may want to implement this.
 - a. Explain the difference between the sliding window method and the block method.
14. Explain the difference between conducting an exhaustive key search vs. English language character frequency.
 - a. Which should help you find a solution faster?
 - b. Explain how your results supported or disproved the number of attempts needed to correctly decrypt the message.
15. Find a character frequency distribution for another language. Provide the frequency distribution in your lab report and be sure to identify which language it is.

Part 02

The ultimate defense against cryptanalysis is choosing a key that is as long as the plaintext and has no statistical relationship to it. A one-time pad is this type of technique. In a one-time pad a message is encrypted by a random key that is as long as the message itself. As discussed in lecture, the key must be completely random with no repeating sequences and the key should never be reused. If used properly, the one-time pad is provably secure.

The activity below demonstrates why a one-time pad is meant to be used only once.

1. Open a terminal and run the following commands. If you get an error about getting low on space, click on the option to empty the trash.

- a. `sudo apt update && sudo apt -y upgrade`
- b. `sudo apt-get install -y python3-numpy`

2. In the terminal you need to generate a random key which is your one-time pad (OTP). You need to give it specific size dimensions so there are enough random bits to XOR with your pictures. The pictures are 500x500. The syntax is `python3`

`generate_pad.py <image_width> <image_height> random_pad.png`

```
cpre3310@cpre3310:~/homework/lab02/part02$ python3 generate_pad.py 500 500 random_pad.png
```

3. Take a screenshot of the image for your lab report. Be sure to capture the image name in the screenshot. These are png files which can be easily opened on the desktop. Navigate to the correct directory (/home/cpre3310/homework/lab02/part02) and click on the image.

4. Now you have to encrypt each image with the same OTP. The image names are puppy.png and heart.png. Be sure to encrypt both images. The syntax is
python3 encrypt_image.py image1.png random_pad.png
encrypted_image1.png

```
cpre3310@cpre3310:~/homework/lab02/part02$ python3 encrypt_image.py puppy.png random_pad.png encrypted_puppy.png
```

5. Take a screenshot of each image for your lab report. Be sure to capture the image names in the screenshot.

6. We now have two files that have been encrypted with the same one time pad. Use the files given to you to XOR the files encrypted_puppy.png and encrypted_heart.png with each other. The output should be saved as leaked_info.png. The syntax is python3 xor_encrypted.py encrypted_puppy.png encrypted_heart.png leaked_info.png

```
cpre3310@cpre3310:~/homework/lab02/part02$ python3 xor_encrypted.py encrypted_puppy.png encrypted_heart.png leaked_info.png
```

7. Take a screenshot of leaked_info.png for your lab report. Ensure the filename is in the screenshot.

8. What do you see in leaked_info.png? Why does this happen? You will want to use the following equations to explain it.

- Plaintext1 $\oplus$ OTP = Ciphertext1
- Plaintext2 $\oplus$ OTP = Ciphertext2

Turn-in

For your lab report, you will be working off of the “Lab 02 Template” that is provided at the end of this document. If you do not use the Lab 02 Template, you will be docked points. After you have completed your lab report, export it as a PDF file, and upload it to Canvas before 11:59pm the night before your lab next week.

Lab 02 Template

Part 01

- 1) **Take a screenshot of your ciphertext character frequencies**
(5 points)
- 2) **Documentation of the iterations to get to the plaintext message. Include a sentence between each iteration explaining the changes.**
(15 points)
- 3) **What is the plaintext message?**
(10 points)
- 4) **What is the key (the mapping of ciphertext character to plaintext character)?**
(10 points)
- 5) **Besides the English language frequency of each character, what else could you have calculated to help you find the plaintext? Explain the difference between the sliding window method and the block method.**
(10 points)
- 6) **Explain the difference between conducting an exhaustive key search vs. English language character frequency. In this lab, which should help you find a solution faster? Explain how your results supported or disproved the number of attempts needed to correctly decrypt the message.**
(15 points)
- 7) **Find a character frequency distribution for another language. Provide the frequency distribution in your lab report *not a link, but the actual distribution) and be sure to identify which language it is.**
(5 points)
- 8) **Submit your commented code as <netid>_lab02_part01.py to Canvas.**
(10 points)

Part 02

- 9) **Include screenshots of random_pad.png, encrypted_puppy.png, encrypted_heart.png, and leaked_info.png in your report.**
(10 points)

10) What do you observe in leaked_info.png? Why does this happen?
(10 points)